MediTrust

Sprint_2_Review Group7

---

1. Sprint 2 Objective

The main objective of Sprint 2 was to extend the MediTrust system by integrating a **machine learning model for cardiovascular risk prediction** into the existing application.

In Sprint 1 we built the core system with frontend pages, backend APIs, authentication, and PostgreSQL database integration. However, the risk calculation used a simple rule-based approach.

During Sprint 2, we focused on replacing this demo logic with a **machine learning model trained on clinical data** and integrating it with the FastAPI backend and frontend interface.

By the end of this sprint, the system should allow:

- Entering detailed patient clinical parameters
- Processing the input through a machine learning model
- Returning a predicted cardiovascular risk level
- Full integration between frontend, backend, and ML components

2. What We Completed in This Sprint

During Sprint 2, we successfully implemented the machine learning pipeline and integrated it with the MediTrust system.

First, we worked on preparing the dataset and building the machine learning pipeline. The cardiovascular dataset was cleaned and preprocessed so it could be used for training models. After preprocessing the data, we trained several machine learning models and evaluated their performance to select a suitable model for prediction.

Once the model training was completed, we integrated the trained model with the FastAPI backend. A new prediction endpoint (/predict) was implemented so that the system can receive patient clinical parameters and return the predicted risk level.

On the frontend side, we improved the risk assessment page so that clinicians can enter multiple clinical parameters such as blood pressure, cholesterol level, ECG results, and other indicators used by the ML model.

The frontend form was then connected to the backend API so that the input data can be sent to the machine learning model for prediction.

This sprint successfully extended the system from a simple prototype to a more realistic AI-based risk prediction tool.

3. Demonstration of Functionality

The following workflow was tested successfully:

1. User logs into the system
2. User opens the risk assessment page
3. Patient clinical parameters are entered in the form
4. The frontend sends the data to the backend /predict API
5. The machine learning model processes the input data
6. The system returns the predicted cardiovascular risk level

The prediction output includes the risk probability and a categorized risk level such as low, medium, or high.

Example output:

Risk Probability: 0.20
Risk Level: Low
Recommendation: Standard evaluation recommended

This confirms that the machine learning model is properly integrated into the system.

(Screenshots of prediction interface, API response, GitHub commits, and project board are attached.)

4. Project Management Tracking

We continued using the GitHub Project Board to track tasks and development progress.

Project Board Link: *https://github.com/users/RavindraSSK/projects/7*

Tasks moved through the following columns:

• Backlog
• Ready
• In Progress
• In Review
• Done

All Sprint 2 tasks were completed and moved to the Done column.

5. Branching Strategy

During development we created several branches in GitHub to organize different tasks and experiments.

However, the main development work for Sprint 2 was completed using the following branches:

main -- sprint2 -- feature/frontend-integration, feature/ml-training

The **feature/frontend-integration** branch was used for frontend interface development, while the **feature/ml-training** branch was used for machine learning model training and backend integration.

Both feature branches were merged into the **sprint2** branch and later integrated into the **main** branch.

Some additional branches such as feature/db-logging, feature/predict-endpoint were created during early planning and experimentation but were not actively used in the final Sprint 2 integration.

6. Challenges Faced

During Sprint 2 we encountered a few technical challenges.

One challenge was ensuring that the frontend input parameters matched the features expected by the machine learning model. Some adjustments were needed to align field names and numeric values so that the backend could process the data correctly.

We also faced some issues while merging Git branches because the local repository was not always synchronized with the remote branches. This caused some frontend changes not to appear initially in the sprint branch. After fetching the latest updates and merging again, the issue was resolved.

Another challenge involved testing the machine learning pipeline and ensuring the predictions were reasonable when integrated with the backend API.

After debugging and testing step by step, we successfully resolved these issues.

We now have:

• A machine learning model integrated with the backend
• A prediction API using FastAPI
• A frontend interface for entering clinical parameters
• Full frontend-to-ML prediction workflow

Note: Respective screenshots are uploaded in screenshot folder